RESEARCH REPORT · OCTOBER 2026

WHO SCORES TONIGHT

Can machine learning pick better goal scorers for the Tim Hortons Hockey Challenge? Three models, three seasons and 99,280 player-games, tested the honest way.



99,280
player-games analysed across 2024–25, 2025–26 and 2026–27

8.1%
better than guessing (log-loss skill) for the new regression model, the best of the three

+44
more scorers in the regression's daily top 25 than v2's, the only pick difference that passed the luck test

Models: tims.py (v1) · tims_v2.py (v2) · train.py Poisson regression · Data: NHL public APIs

AT A GLANCE

The short version

We built two new ways to predict who scores in an NHL game and tested them against the original model on two and a half seasons of real games, using only information that was available on the morning of each game.

1. The machine-learning model gives the most accurate percentages.

A Poisson regression trained on past games beat both hand-built models on every accuracy measure, on seasons it never saw: log loss 0.3908 vs 0.3916 (v2) and 0.3939 (v1). When it says 30%, players score about 30% of the time.

2. It picks better players in the top 25, but not clearly in the top 10.

Over 186 out-of-sample game days, the regression's daily top 25 had +44 more scorers than v2, and that gain held up in a luck test (95% range +12 to +74). Every top-10 difference between the three models falls inside the range luck alone produces.

3. Ice time is the single biggest signal.

The model's two largest weights are power-play minutes and even-strength minutes. Players expected to get 3.5+ power-play minutes scored 30% of the time, against 7% for players with almost none. Opponent strength, home ice and back-to-backs matter much less than most people assume.

4. There's a hard ceiling, and it's luck.

Even the best model is only about 8% better than guessing everyone's chance at the league average. About 19% of forwards score in any given game, and the best picks top out around 35–45%. No model will make goal scoring predictable; the realistic win is a few extra correct picks per month.

Recommendation

Use the **regression for the percentages shown on the site**, since they're the most trustworthy, and for ranks 11–50. For the one or two daily Tims picks, all three models are statistically tied, so the regression is a safe default but not a guaranteed upgrade. Re-run the training each season as more data arrives.

Contents

1 The challenge and why it's hard

2 How the models are judged

3 The data and the no-peeking rules

4 Model v1: the original formula

5 Model v2: splitting out the power play

6 Model 3: the Poisson regression

7 Results
8 What the model learned
9 A day in detail: Oct 17, 2026
10 Limits, caveats and next steps
A Appendix: files, commands, glossary

PART 1

The challenge and why it's hard

In the Tim Hortons Hockey Challenge you pick players before the games; if one scores, you win. So the question every model answers is simple: **what is this player's chance of scoring at least one goal tonight?**

19.1%
of forwards score in a game they play

7.0%
of defencemen do

11%
of players who score get two or more

Why it's a hard prediction problem

- **Goals are rare.** Across all 99,280 player-games, only about 15% ended with a goal. Most predictions are "probably not."
- **Goals are noisy.** A great shooter takes 3–4 shots a night, and each one goes in maybe 1 time in 7. Posts, goalies and deflections decide a lot.
- **The edges are small.** The difference between a 40% player and a 35% player is real, but over one night it's mostly invisible. You need hundreds of days to see it.
- **It's easy to fool yourself.** A model can look brilliant on the games it was built from and fall apart on new ones. Part 3 covers how this research avoids that.

What "good" looks like

A useful model needs to do two things well:

1. **Rank players**, so the top picks each day score more often than average picks. This is what wins Tims.
2. **Give honest percentages**, so "40%" really means about 4 in 10. This is what makes the website trustworthy.

These are different skills. A model can rank well and still say 60% when the truth is 40%, or have perfect percentages but bunch everyone together. The research measures both.

The math every model shares

All three models first estimate **expected goals (xG)** for the night: the average number of goals the player would score if this game were replayed many times. They then turn it into a chance of scoring at least once, treating goals as random events (a Poisson process):

```
P(score at least once) = 1 − e^(−xG)

xG 0.20 → 18%     xG 0.40 → 33%     xG 0.60 → 45%
```

The models differ only in how they estimate xG.

PART 2

How the models are judged

Five measures, each answering a plain question. You don't need the math to read the results; the arrows in every table show which direction is better.

LOG LOSS ↓ · "HOW SURPRISED WAS THE MODEL?"

For every player-game, the model is penalized by how surprised it was by what happened. Saying 5% for a player who then scores is a big penalty; saying 40% for a player who scores is a small one. Lower is better. This is the main accuracy score, and it's the one the regression was trained to minimize.

SKILL ↑ · "HOW MUCH BETTER THAN GUESSING?"

Log loss turned into a percentage: how much better the model is than guessing every player at the league-average rate (about 15%). 0% means useless, and 100% would mean perfect foresight, which is impossible in hockey. A skill of 8% is genuinely good for this problem.

BRIER SCORE ↓ · "AVERAGE SQUARED MISS"

A second accuracy measure: the average of (predicted chance − what happened)^2. It tells the same story as log loss and is included as a cross-check.

TOP N ↑ · "DID THE PICKS SCORE?"

Each day, take the model's N highest-ranked players and count how many scored. "Top 10: 38%" means that on an average day, about 3.8 of the model's top 10 scored. This is the measure closest to winning Tims.

CALIBRATION · "DOES 30% MEAN 30%?"

Group players by their predicted chance (say, everyone the model gave 25–30%) and check what share actually scored. A well-calibrated model's points sit on the diagonal line in the calibration chart.

The luck test (bootstrap)

When two models differ by, say, 15 correct picks over a season, is that skill or luck? The test resamples the game days 2,000 times with replacement and recalculates the difference each time. If 95% of those reshuffled seasons still favour the same model, the gap is unlikely to be luck. If the 95% range includes zero, we can't tell the models apart, and the report says so plainly.

Fair comparisons

- **Same players, same days.** Every model is scored on exactly the same player-games.
- **Only players who dressed.** Each game's box score defines who played, so a scratched player can't count against a model. This also places every player on the team he actually played for that night, which matters after trades.
- **Ranked by predicted chance.** Top-N counts use each model's own ordering for that day.

PART 3

The data and the no-peeking rules

Everything comes from the NHL's free public APIs. The hard part isn't collecting the data, it's making sure every prediction uses only what was knowable on the morning of the game.

1 · Schedule
Which games are on each date

→

2 · Box scores
Who dressed, and for which team

→

3 · Stats as of that morning
Ice time, shots, goals, standings, penalty kill

→

4 · Predict
v1 and v2 compute their xG; each row is saved

→

5 · Score
Compare with who actually scored

Sources

Data	Where it comes from	Used for
Player profiles and game logs	api-web.nhle.com	Shots, goals, position, recent form, missed games
Even-strength / power-play time and shots	api.nhle.com/stats (skater reports)	v2 and regression ice time and shot rates
Standings (day before)	api-web.nhle.com/standings	Opponent goals against
Team penalty kill	api.nhle.com/stats (team reports)	Opponent PP goals allowed, times shorthanded
Box scores and scores	api-web.nhle.com/gamecenter, /score	Who dressed, who scored (shootout goals excluded)

The no-peeking rules

- **Stats stop the day before.** Every season-to-date number is filtered to games before the target date. Standings are taken from the previous day.
- **Previous seasons are fair game.** Full prior-season totals were known before the season started.
- **Seasons are split by time, never at random.** The regression learned from 2024–25, was tuned on 2025–26, and was tested once on 2026–27. A random split would let it learn from games later than the ones it predicts.

The dataset

Season	Game days	Player-games	Scored	Role for the regression
2024–25	178	47,225	14.9%	Training: learns the weights
2025–26	167	47,231	15.2%	Validation: chooses settings, then joins training
2026–27 (to Oct 17)	19	4,824	14.9%	Test: scored once, at the very end



The test bar is drawn three times wider than its share so it's visible; it holds 4,824 player-games.

PART 4

Model v1: the original formula

The model the website runs today (tims.py). It's hand-built from hockey intuition: take a player's shot volume, multiply by how often his shots go in, and adjust for the matchup.

```
xG = shots per game × shooting % × opponent factor × home/away factor
```

THE INGREDIENTS

1. **Shots per game** over this season and the previous two, weighted 1.0 / 0.6 / 0.3. Shot volume is the most stable trait a shooter has.
2. **Recent form:** the last 10 games' shot rate gets up to 30% of the weight, catching line changes.
3. **Shooting %** blended with 100 imaginary league-average shots (10.5% forwards, 4.5% defence), so a hot start on few shots can't dominate.
4. **Opponent:** goals against per game vs league average, shrunk toward average early in the season and capped at 0.8–1.25×.
5. **Home ice:** +3% at home, −3% away.

```
Sh%  = (weighted goals + 0.105 × 100) / (weighted shots + 100)     forwards
OppF = clamp( (GA + 10 × league GA/GP) / (GP + 10) / league GA/GP , 0.8, 1.25)
P    = 1 − e^(−xG)
```

Strengths and blind spots

v1 is simple, transparent and surprisingly strong; it's still tied for the best top-10 picks. Its blind spot is that it treats every minute alike. A shot on the power play is far more likely to be a goal, and a player's role (first-unit PP or fourth line) only shows up indirectly through his shot totals.

PART 5

Model v2: splitting out the power play

tims_v2.py scores even strength (EV) and the power play (PP) separately, and builds xG from expected ice time instead of games.

```
xG = venue × [ EV min/60 × EV shots/60 × EV sh% × opp EV defence
            + PP min × opp penalties/60 × PP shots/60 × PP sh% × opp penalty kill ]
```

WHAT CHANGED

- **Ice time:** EV and PP minutes per game, weighted toward this season (1.0 / 0.4 / 0.1) and pulled up to 50% toward the last two weeks, so a promotion to the top power-play unit shows up within days.
- **Per-minute rates:** EV and PP shots per 60 minutes, each shrunk toward the league rate for the position (120 and 30 prior minutes).
- **Separate finishing:** EV and PP shooting %, each shrunk toward the league rate (100 and 50 prior shots).
- **Smarter matchups:** the opponent's even-strength goals allowed for the EV part; for the PP part, how often they take penalties (more PP minutes) and how good their penalty kill is.
- **League rates computed, not guessed:** each day uses last season's actual league totals by position.

Result in one line: v2's percentages are more accurate than v1's in every season, but its top-10 picks aren't better.

PART 6

Model 3: the Poisson regression

Instead of a person choosing the formula's numbers (why 0.6 for last season? why 3% for home ice?), the regression learns them from 94,456 real player-games.

The idea in plain terms

v1 and v2 multiply factors together. Take the logarithm and a product becomes a sum, which is exactly what a regression handles:

```
log(xG) = b0 + b1·log(PP minutes) + b2·log(EV minutes) + … + b_home·home + …
xG      = e^(that sum)          P(score) = 1 − e^(−xG)
```

Each weight **b** says how strongly one input moves tonight's xG. The model starts with every weight at zero and adjusts them until its predictions best match who actually scored, measured by the Poisson likelihood of each player's goal count. The fit uses Newton's method and converged in 6 steps (about 0.06 seconds with NumPy).

The 20 inputs

Every ingredient from v1 and v2, plus a few new signals, all computed as of that morning:

FROM V2

- EV and PP minutes
- EV and PP shots per 60
- EV and PP shooting %
- Opponent EV defence, penalties, penalty kill

FROM V1

- Shots per game (with recent form)
- Shooting %
- Opponent goals against

NEW

- EV and PP minutes **trend** (last two weeks vs season)
- Games played this season
- Missed games in a row
- Home / away
- Defenceman or forward
- Team on a back-to-back
- Opponent on a back-to-back

Guarding against overfitting

A model with many inputs can memorise quirks of its training season. Two safeguards:

- **Regularization (L2 penalty λ):** a cost for large weights, which keeps the model cautious. Five strengths were tried, each trained on 2024–25 and scored on 2025–26:

λ	Validation log loss ↓
0.1	0.39129
1	0.39129
10	0.39126
100 (chosen)	**0.39124**
1000	0.39131

The differences are tiny, which is good news: with ~47,000 training rows and 20 inputs, the model isn't overfitting much at any setting.

- **A sealed test season:** 2026–27 was never used for any choice. Its numbers are reported once, and they're the most honest estimate of real-world performance.

PART 7

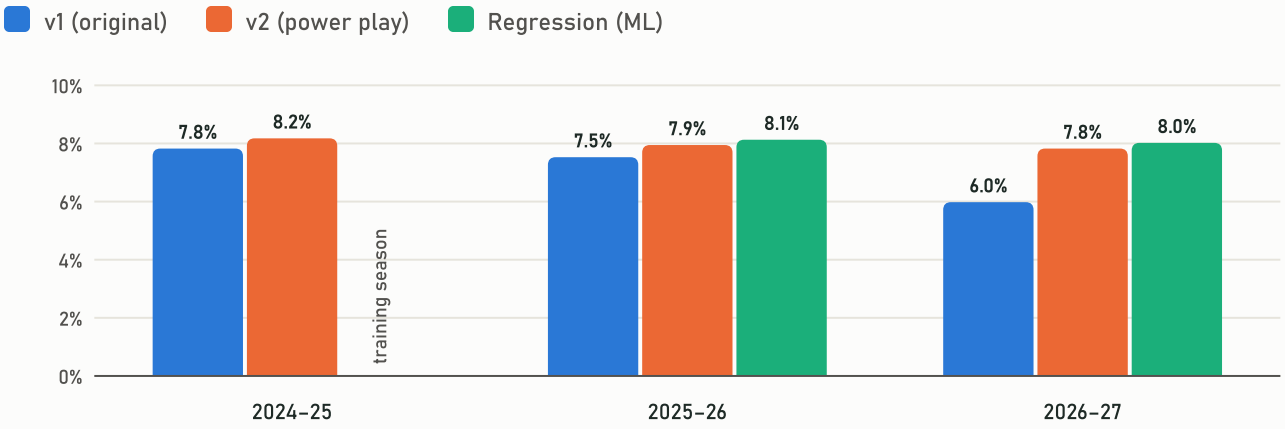
Results

The full scoreboard, season by season. Bold marks the best model in each season for log loss, top 10 and top 25.

Season	Model	Log loss ↓	Brier ↓	Skill ↑	Top 10 ↑	Top 25 ↑	Top 50 ↑
2024–25	v1 (original)	0.3886	0.1183	7.8%	36.2%	32.2%	28.0%
2024–25	v2 (power play)	**0.3872**	0.1182	8.2%	**36.5%**	**32.4%**	28.3%
2024–25	Regression (ML)	Training season (the regression learned from it, so scoring it would be cheating)					
2025–26	v1 (original)	0.3938	0.1201	7.5%	**38.6%**	34.5%	29.7%
2025–26	v2 (power play)	0.3920	0.1199	7.9%	37.5%	33.9%	30.0%
2025–26	Regression (ML)	**0.3912**	0.1197	8.1%	38.0%	**34.7%**	29.7%
2026–27	v1 (original)	0.3952	0.1186	6.0%	36.3%	31.6%	27.4%
2026–27	v2 (power play)	0.3875	0.1183	7.8%	**37.9%**	31.2%	27.8%
2026–27	Regression (ML)	**0.3866**	0.1180	8.0%	36.3%	**33.5%**	28.0%

Accuracy: how much better than guessing

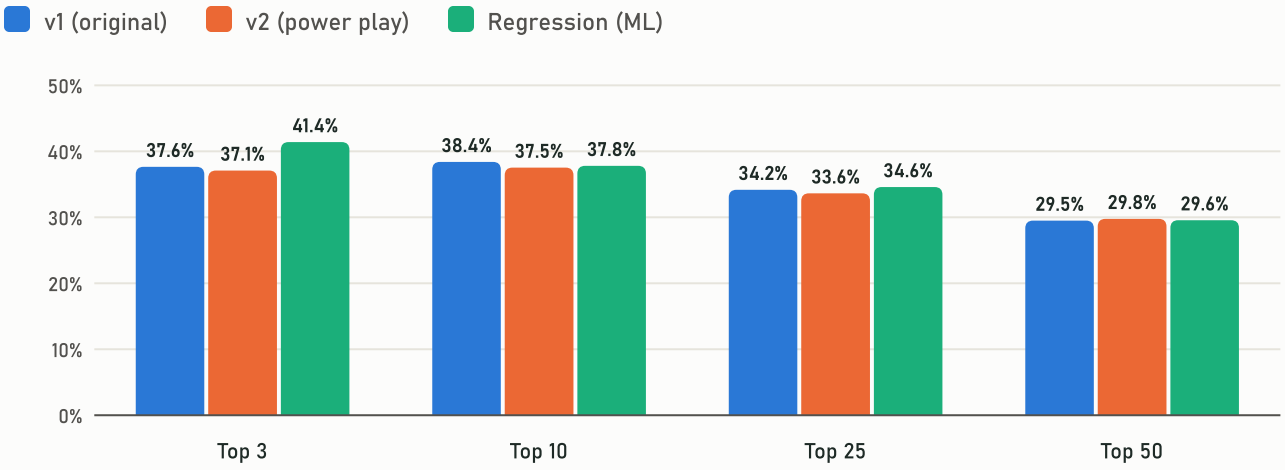
Log-loss skill vs predicting the league-average rate for everyone. Higher is better.



The regression has the highest skill in both seasons it can be fairly scored on. 2024–25 is the regression's training season, so it isn't scored there. v2 beats v1 in every season.

Picks: share of each day's top players who scored

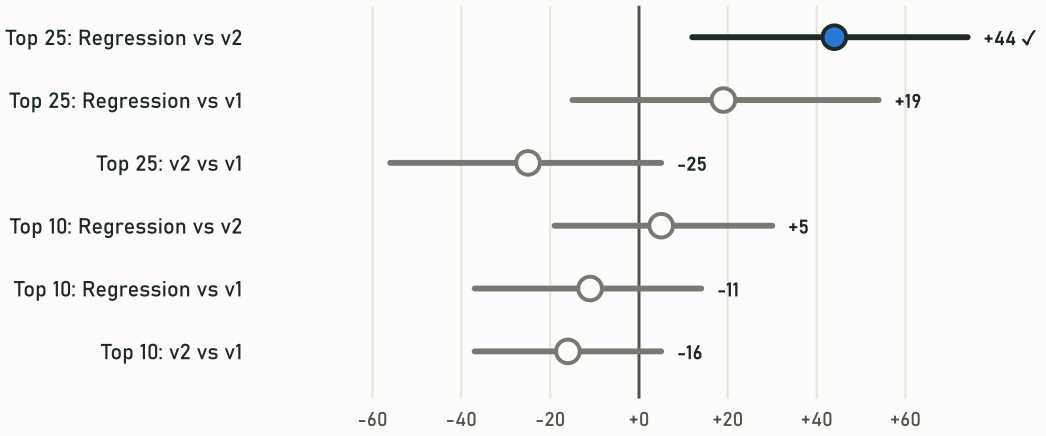
Out of sample: 2025–26 and 2026–27 combined (186 game days). Higher is better.



All three models are close. The regression leads at the top 3 (41.4%) and top 25 (34.6%); v1 leads narrowly at the top 10 (38.4%). For comparison, a random dressed skater scores about 15% of the time.

The luck test: extra scorers in the daily picks

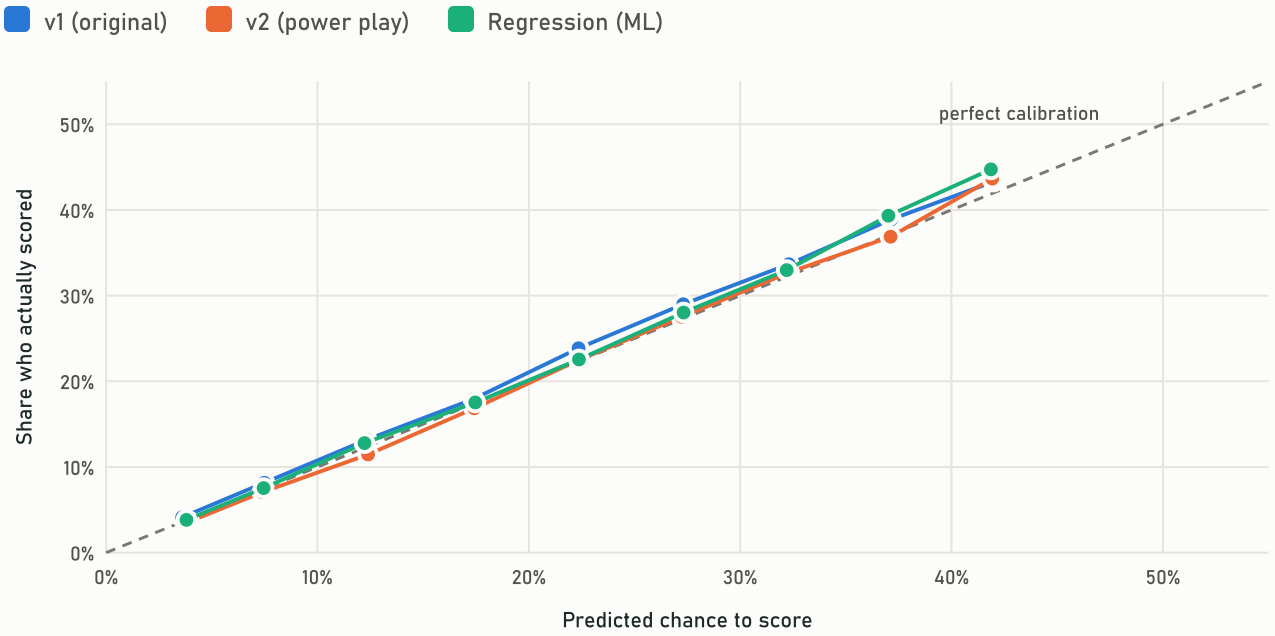
Difference in total scorers over 186 days, with the 95% range from 2,000 reshuffled seasons. ✓ = the range excludes zero, so it's unlikely to be luck.



Only one gap clears the bar: the regression's top 25 beat v2's by +44 scorers. Every top-10 difference, including v1's lead, sits inside the luck range.

Calibration: does a predicted 30% really mean 30%?

Out of sample (2025–26 and 2026–27). Players grouped into 5-point bands of predicted chance; bands with fewer than 100 players are left out.



Up to 45%, every model is within about 2 points of the line. Weighted by players, the regression is off by an average of 0.35 points, v2 by 0.5 and v1 by 0.9 (v1 runs slightly low, so it's a little too cautious). Almost no player is ever given more than 45%, so the chart stops there.

Reading the results together

- **Accuracy:** regression > v2 > v1, consistently and by a margin that holds across seasons.
- **Top 10:** a statistical tie. Better percentages don't automatically produce better top picks, because the top of the list is a small group of elite shooters every model agrees on. The regression's top-3 lead (41% vs 38%) looks promising, but it rests on too few picks to rule out luck.
- **Ranks 11–25:** the regression's one clear win (over v2). It sorts the second tier of players better, where ice time separates players whose shot totals look alike.

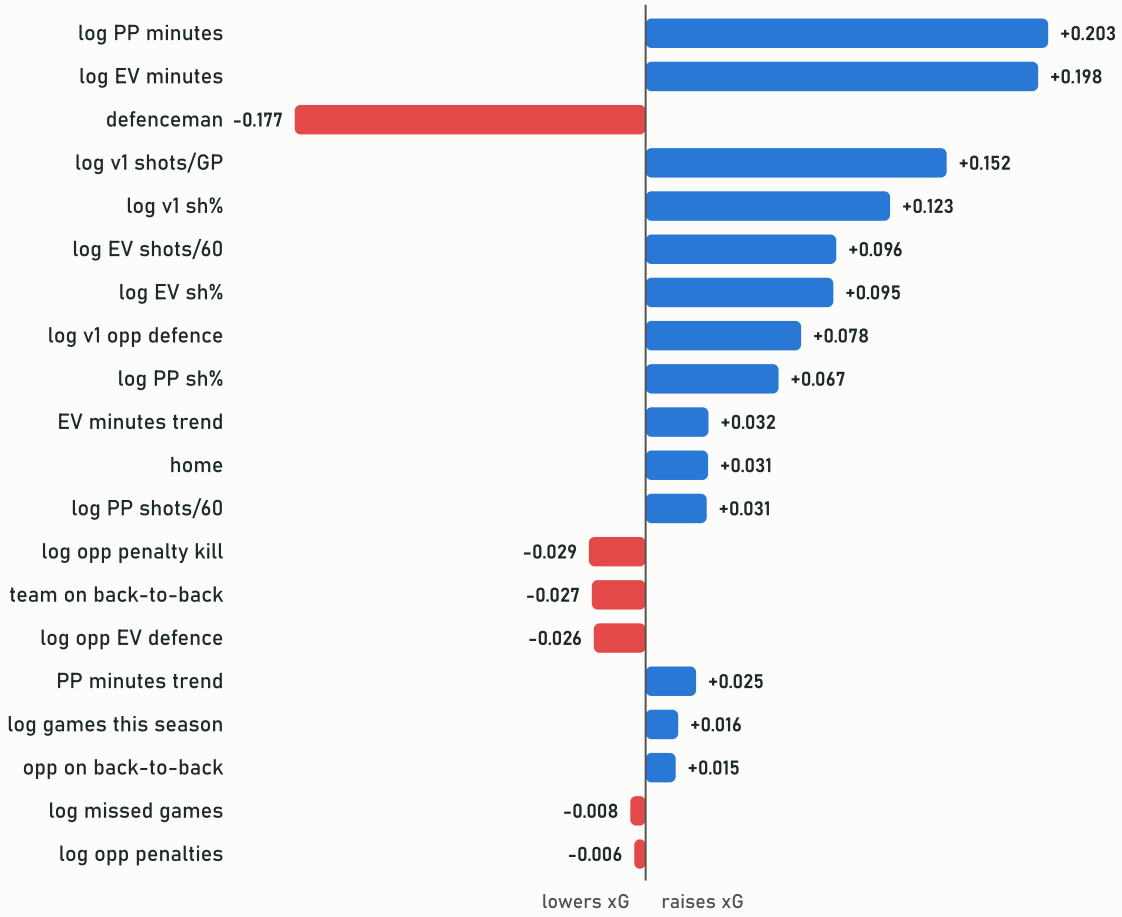
PART 8

What the model learned

Because every input is standardized (rescaled so a "typical" difference is 1), the weights can be compared directly. A bigger bar means that input moves a player's chances more.

Regression weights

Trained on 2024–25 and 2025–26. Blue raises expected goals, red lowers them.



Five takeaways

1. **Ice time is king.** Power-play minutes (+0.20) and even-strength minutes (+0.20) are the two biggest weights. Who the coach puts on the ice matters more than anything else the model sees.
2. **Defencemen are discounted heavily** (−0.18), even after accounting for their ice time and shots, most likely because their shots come from farther out.
3. **Shot volume and finishing still matter** (v1's shots/GP +0.15, shooting % +0.12), so the original formula's core idea was right.

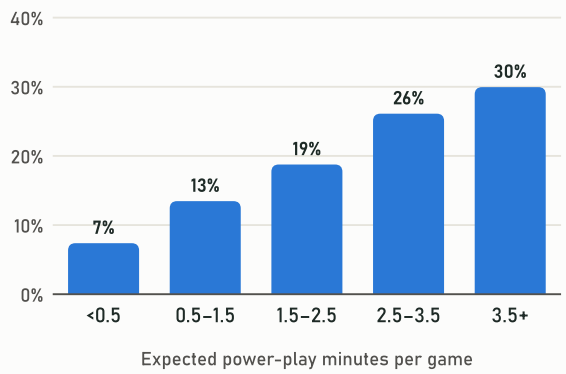
4. **Matchups are a small effect.** The opponent inputs are small and partly cancel each other out (v1's and v2's opponent measures overlap, so the model splits the credit). Combined, a weak defence helps, but less than ice time does.
5. **Schedule effects are minor.** Home ice (+0.03) and back-to-backs (−0.03 for the player's team) are real but small. Over the data, home players scored 15.5% vs 14.6% away, and players on a back-to-back 14.2% vs 15.2% rested.

Ice time in the raw data

The same story shows up without any model at all: scoring rate by the minutes a player was expected to get, across all 99,280 player-games.

Power-play minutes

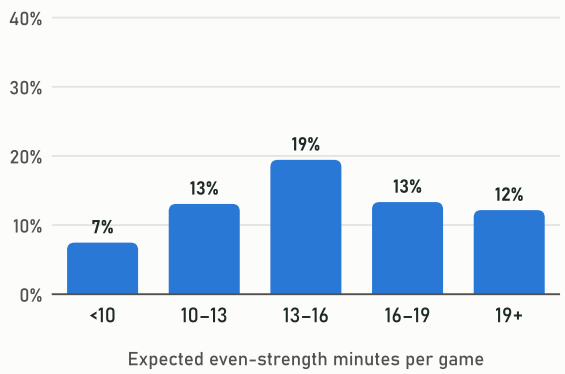
Share who scored



A steady climb, from 7% to 30%. First-unit power-play players score about four times as often.

Even-strength minutes

Share who scored



Peaks at 13–16 minutes and then falls. The heaviest-minute players are mostly defencemen, which is why the model needs the defenceman input.

Why the even-strength chart dips

This is a classic trap in sports data: **confounding**. Big even-strength minutes look bad for scoring only because defencemen, who score rarely, play the most. Once the model knows who's a defenceman, more minutes help everyone. Raw averages can mislead; a model that weighs everything at once untangles them.

PART 9

A day in detail: Oct 17, 2026

The regression's top 12 on the last day of the test season, next to where v1 ranked the same players. Expected EV and PP minutes are per game.

Reg. rank	Player	Game	EV min	PP min	Regression	v1 rank	v1	Result
1	Cole Caufield	MTL vs BUF	15.9	3.6	40%	2	43%	**Scored**
2	Alex DeBrincat	DET vs SJS	15.5	4.0	40%	5	41%	**Scored**
3	Nikita Kucherov	TBL vs VAN	16.4	4.7	39%	4	42%	**Scored**
4	Kyle Connor	WPG vs CAR	16.8	3.1	39%	7	39%	–
5	Auston Matthews	TOR vs NYI	17.3	3.5	39%	14	35%	–
6	Leon Draisaitl	EDM vs UTA	18.0	3.4	39%	8	39%	**Scored**
7	Brandon Hagel	TBL vs VAN	15.2	3.6	38%	1	47%	–
8	Tage Thompson	BUF vs MTL	16.3	3.4	38%	3	42%	**Scored**
9	William Nylander	TOR vs NYI	17.0	3.5	38%	13	35%	–
10	Connor McDavid	EDM vs UTA	19.0	3.4	37%	22	33%	**Scored**
11	Filip Forsberg	NSH vs STL	16.7	3.8	37%	19	34%	**Scored**
12	Jason Robertson	DAL vs CHI	16.4	3.3	36%	9	37%	**Scored**

What this day shows

- **The models mostly agree on who's elite.** Ten of the regression's top 12 were also in v1's top 14.
- **The regression trusts ice time more.** Connor McDavid (19.0 EV and 3.4 PP minutes) was 22nd for v1 but 10th for the regression, and he scored. Auston Matthews and William Nylander moved up the same way.
- **It's less swayed by hot starts.** Brandon Hagel led v1 at 47% after a 6-goals-in-7 start; the regression had him 7th at 38%. He didn't score, but one day proves nothing: that's what the luck test is for.
- **This was an unusually good day.** 8 of these 12 scored. Over the long run, a model's daily top 10 has only about 3 or 4 scorers (see the picks chart), so most days look far less impressive.

PART 10

Limits, caveats and next steps

Caveats

- **The test season is short.** 2026–27 has only 19 game days so far. Its numbers agree with the much larger 2025–26 validation season, but they'll move as the season goes on.
- **Only dressed players are scored.** In real Tims picks you choose before line-ups are confirmed. All three models face the same scratch risk, so the comparison is fair, but real hit rates will be slightly lower.
- **Late-season context isn't modelled.** Teams resting players, call-ups and trade-deadline moves are only captured through ice-time trends.
- **Small effects are hard to measure.** With hockey's randomness, differences of a few picks per season can't be proven either way.

Next steps, in order of expected value

1. **Put the regression on the site.** The weights in model.json can be applied in plain Python inside tims.py, with no new libraries. The site would show the most accurate percentages available.
2. **Add linemates and team offence.** Playing with an elite centre, or for a team that scores a lot, likely matters and isn't captured yet.
3. **Add the opposing goalie.** Starters are announced in the morning; a backup goalie on a back-to-back is a meaningful edge.
4. **Try gradient-boosted trees (LightGBM).** They can learn interactions, like PP time mattering more against a weak penalty kill. They're worth testing once the simpler upgrades are in, using the same backtest.
5. **More seasons.** Adding 2022–23 and 2023–24 roughly doubles the training data and makes small effects measurable.

About the GPU

train.py will use the RTX 5070 automatically if PyTorch with CUDA is installed. For this model it isn't needed: the whole fit takes about 0.06 seconds on the CPU, faster than a GPU can start up. The GPU becomes useful for larger models such as neural networks or many more inputs.

APPENDIX

Files, commands and glossary

Files

File	What it is
tims.py	v1, the live model the website uses
tims_v2.py	v2: even-strength / power-play split and ice time
backtest.py	Re-ranks past days with v1 and v2 and scores them; --csv saves training data
train.py	Fits the Poisson regression, compares all three models, saves model.json
training_data.csv	99,280 player-games, about 35 inputs each, plus the result
model.json	The fitted model: input names, scaling, weights and scores

Commands

```
py backtest.py --season 20242025 20252026 20262027 --csv training_data.csv
py train.py training_data.csv
py backtest.py 2026-09-29           # quick v1 vs v2 check on this season
```

All regression weights

#	Input	Weight	What it measures
1	log PP minutes	+0.203	Expected power-play minutes tonight (season average pulled toward the last two weeks).
2	log EV minutes	+0.198	Expected even-strength minutes tonight.
3	defenceman	-0.177	1 for defencemen, 0 for forwards.
4	log v1 shots/GP	+0.152	v1's shots per game: three seasons weighted 1.0/0.6/0.3, nudged by the last 10 games.
5	log v1 sh%	+0.123	v1's shooting %, blended with 100 league-average shots.
6	log EV shots/60	+0.096	v2's even-strength shots per 60 minutes, shrunk toward the league rate.
7	log EV sh%	+0.095	v2's even-strength shooting %, shrunk toward the league rate.
8	log v1 opp defence	+0.078	Opponent's goals against per game vs league (v1's version).
9	log PP sh%	+0.067	Power-play shooting %, shrunk toward the league rate.
10	EV minutes trend	+0.032	Last two weeks' EV minutes compared with the season average.
11	home	+0.031	1 at home, 0 away.
12	log PP shots/60	+0.031	Power-play shots per 60 minutes.
13	log opp penalty kill	-0.029	Opponent's PP goals allowed per time shorthanded vs league.
14	team on back-to-back	-0.027	1 if the player's team also played yesterday.
15	log opp EV defence	-0.026	Opponent's even-strength goals allowed per game vs league (v2's version).
16	PP minutes trend	+0.025	Last two weeks' PP minutes compared with the season average.
17	log games this season	+0.016	Games the player has played so far this season.
18	opp on back-to-back	+0.015	1 if the opponent also played yesterday.
19	log missed games	-0.008	Team games missed in a row before tonight (injury or scratch).
20	log opp penalties	-0.006	How often the opponent is shorthanded per game vs league.

Glossary

xG	Expected goals: the average goals a player would score if tonight were replayed many times.
EV / PP	Even strength (equal players on the ice) / power play (the other team has a penalty).
Shrinkage	Blending a small sample with the league average so early-season flukes don't dominate.
Poisson regression	A model for counts (like goals) that learns how each input multiplies the expected count.
Standardized	Rescaled so each input has an average of 0 and a typical spread of 1, which makes weights comparable.
Regularization (λ)	A penalty on large weights that keeps the model from memorising its training data.
Out of sample	Games the model never learned from. The only honest place to judge it.
Bootstrap	Resampling the game days many times to see how much a result could move by luck.
Calibration	Whether predicted chances match how often things actually happen.